

1. Uso de XTTSv2 en Kaggle

Para utilizar este TTS, copie directamente la plantilla ya preparada: [PlantillaOptimizada XTTS-coqui](#) (botón *Copy & Edit* de Kaggle). A diferencia de una versión anterior de este documento, ya **no** es necesario escribir ni editar el código de entrenamiento: la plantilla entrena, optimiza (poda de capas + recuperación), valida y empaqueta el modelo automáticamente en una sola corrida. Solo hace falta preparar el dataset, subirlo, y editar una única celda de configuración.

1.1. Creación de una cuenta y copia de la plantilla

Se utilizará Kaggle como entorno en la nube para entrenar/generar el modelo TTS ya que con el plan gratuito ofrece 30h semanales de uso en GPUs y persistencia en el almacenamiento (20GB).

Para crear una cuenta puede dirigirse a la página oficial [Kaggle](#) y registrarse con sus datos. Es necesario que introduzca su número de teléfono para poder acceder a las GPUs de Kaggle, ya que se necesitan para poder entrenar los modelos de voz.

Una vez iniciada la sesión, abra el enlace de la plantilla anterior y presione *Copy & Edit*. Esto crea una copia propia del notebook, con todas las celdas del pipeline ya escritas – no hay una celda vacía por defecto que borrar ni celdas de instalación que redactar a mano.

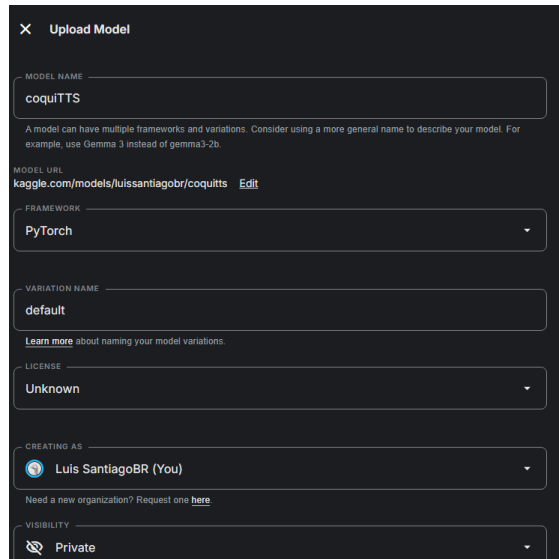
1.2. Subida de archivos a Kaggle

A diferencia de otras aplicaciones, Kaggle solo permite subir archivos como un *dataset* o *model*. Para lograr esto, en el menú desplegable de la derecha del editor del notebook están las opciones *Add Input* (para usar archivos ya cargados anteriormente por nosotros u otra persona) o *Upload* (para nuevos archivos).

Nota: Para el dataset es posible subir un archivo `.zip` el cual se descomprimirá automáticamente.

Para el entrenamiento del TTS, el dataset debe contener las grabaciones y transcripciones, de una voz seleccionada en un formato específico (explicado más adelante).

La plantilla necesita un *Model* de Kaggle llamado `coquitts`, que empaqueta el código de coquiTTS para instalarlo directamente sin depender de PyPI. Este *Model* se crea UNA sola vez: impórtelo desde el repositorio oficial <https://github.com/idiap/coqui-ai-TTS> usando la configuración de la Figura 1, y luego añádalo como *Input* en cualquier copia de la plantilla que use.



Upload Model

MODEL NAME
coquiTTS

A model can have multiple frameworks and variations. Consider using a more general name to describe your model. For example, use Gemma 3 instead of gemma3-2b.

MODEL URL
kaggle.com/models/luissantiagobr/coquits [Edit](#)

FRAMEWORK
PyTorch

VARIATION NAME
default

[Learn more](#) about naming your model variations.

LICENSE
Unknown

CREATING AS
Luis SantiagoBR (You)

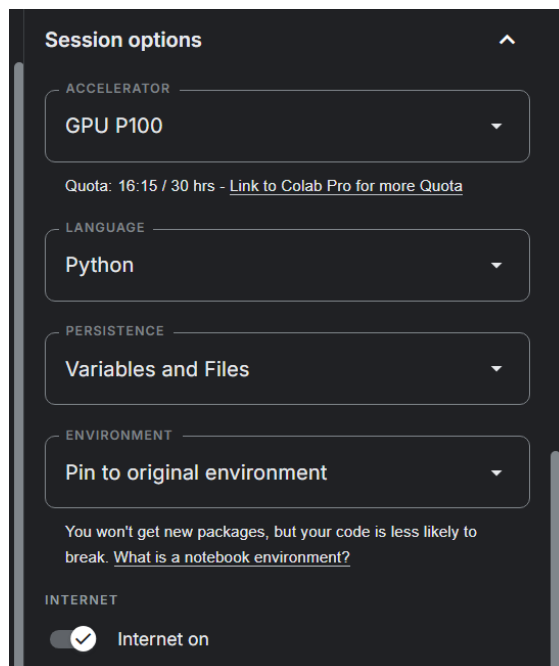
Need a new organization? [Request one here](#).

VISIBILITY
Private

Figura 1: Datos sobre el repositorio.

1.3. Configuración de la sesión

Hay ajustes modificables en Kaggle, entre ellos el *Acelerador* (CPU/None, GPU, TPU), la persistencia de almacenamiento y el uso de internet; para modificarlos, en el mismo menú de la derecha puede desplazarse hacia abajo hasta encontrar los ajustes de la Figura 2.



Session options

ACCELERATOR
GPU P100

Quota: 16:15 / 30 hrs - [Link to Colab Pro for more Quota](#)

LANGUAGE
Python

PERSISTENCE
Variables and Files

ENVIRONMENT
Pin to original environment

You won't get new packages, but your code is less likely to break. [What is a notebook environment?](#)

INTERNET
☒ Internet on

Figura 2: Ajustes de la sesión.

Para el modelo a usar (XTTS - coquiTTS), esta plantilla está configurada y validada para **GPU T4**. El código fija internamente el uso de una sola GPU (`CUDA_VISIBLE_DEVICES`), así

que no hay ningún beneficio en seleccionar $T4\ x2$: solo una de las dos se usará. El acelerador puede modificarse después (lo que reiniciará el entorno).

Con esto ya se encuentra configurado el entorno para iniciar una sesión (de máximo 12h seguidas) dando click al botón de “encender”.

1.4. Preparar el dataset

Para el entrenamiento se requieren las grabaciones en formato `.wav` y todas las transcripciones en un mismo archivo `.csv` siguiendo la estructura:

```
dataset
|
+-- metadata.csv
+-- wavs
    |
    +-- example1.wav
    +-- example2.wav
```

Con el fin de automatizar el proceso de organización del dataset (a partir de las grabaciones y transcripciones de la base de datos del TCU), descargue y corra localmente el script `generar_dataset.py` y modifique cerca del final la línea que corresponde a la ruta del dataset original.

```
input_dir = "RUTA_DATASET"
```

En caso de diferencias en los formatos de los archivos, puede modificar la extensión de los audios o el encoding de las transcripciones en las variables correspondientes. Una vez generado, comprima la carpeta `dataset` resultante y súbala a Kaggle como *Dataset* (ver sección anterior).

1.5. Configuración y ejecución de la plantilla

Dentro del notebook, la única celda que debe editarse es la que escribe `config.py` (identificada por el comentario `OBLIGATORIO`). El resto de celdas ya contienen el código de instalación de dependencias y del pipeline completo – no requieren edición, solo ejecutarse en orden.

Importante: debe completar, como mínimo, las siguientes variables de esa celda:

```
VOICE_NAME = "NOMBRE_DE_LA_VOZ"
LANGUAGE = "es"

# Dataset de Kaggle subido en el paso anterior:
DATASET_OWNER = "SU_USUARIO_DE_KAGGLE"
DATASET_SLUG = "NOMBRE_DEL_DATASET"

# Archivo de audio DENTRO del dataset a usar como referencia de la voz
# (solo el nombre del archivo, no una ruta completa):
```

```

SPEAKER_REFERENCE_FILENAME = "ARCHIVO_DE_REFERENCIA.wav"

# Transcripción EXACTA de ese archivo de referencia -- se usa para
# validar automáticamente el audio generado durante y al final del entrenamiento:
TEST_SENTENCE = "Transcripción exacta del audio de referencia."

# Cuenta de Kaggle donde subió el Model "coqutts" (ver sección anterior):
COQUITTS_MODEL_OWNER = "SU_USUARIO_DE_KAGGLE"

```

Todo lo demás (hiperparámetros de entrenamiento, poda de capas, criterio de parada automática, validaciones de audio) ya tiene valores por defecto probados y no necesita tocarse. Solo en caso de querer optimizar una voz ya entrenada previamente (en vez de entrenar una nueva desde cero) debe además poner `SKIP_INITIAL_FINETUNE = True` y completar `EXISTING_CHECKPOINT_PATH/EXISTING_CONFIG_PATH` con la ruta de ese checkpoint (subido como Dataset o Model).

Con la configuración lista, siga las instrucciones de la primera celda del notebook: **Save Version - Save & Run All - Run with GPU**. Esto ejecuta, en orden y sin intervención manual: instalación de dependencias, preparación del dataset, entrenamiento inicial, poda de capas, entrenamiento de recuperación, validación automática del audio en cada paso, exportación a ONNX y empaquetado final. No hace falta vigilar el progreso ni decidir cuántas épocas entrenar – el proceso se detiene solo, ya sea porque el modelo dejó de mejorar (convergencia) o porque llegó a un tope de tiempo de seguridad.

1.6. Nota Importante

Kaggle no permite inactividad del navegador por más de 40min y cierra la sesión. Por ello es indispensable usar *Save Version -& Save & Run All* (en vez de correr las celdas manualmente): esto ejecuta el notebook en segundo plano en la infraestructura de Kaggle, sin depender de que el navegador permanezca abierto.

Puede dirigirse a <https://www.kaggle.com/work> y buscar su notebook: en la pestaña *Logs* podrá ver el progreso del entrenamiento en tiempo real y detenerlo si lo desea; en la pestaña *Output* podrá descargar el resultado, una carpeta `output/final_voice/<voz>_<fecha>` que contiene:

- `models/best_model.pth`, `models/config.json`, `models/vocab.json`, `models/ref.wav`
- `onnx_models/hifigan_decoder.onnx`
- `manifest.json` – con las instrucciones exactas de qué copiar a VocesWindows/ (o VocesLinux/) y cómo renombrarlo.

Esta plantilla no reproduce ni sintetiza voz de forma interactiva dentro de Kaggle – solo entrena y prepara el modelo final. Una vez copiados esos archivos a VocesWindows/VocesLinux/ siguiendo el `manifest.json`, la voz queda lista para generar audio ahí, de forma local y sin GPU.

Documentación Original[?]: <https://docs.coqui.ai/en/latest/>

Documentación Comunidad[?]: <https://coqui-tts.readthedocs.io/en/latest/>

2. Anexos

2.1. Errores comunes entrenando voces con XTTS en Kaggle

2.1.1. Error de memoria insuficiente

Un error que puede suceder es que no se haya seleccionado la GPU como acelerador del notebook, por lo tanto, se llena la memoria y por ende, correrá por unas horas antes de fallar. Como consecuencia, se obtiene una pestaña del notebook en la sección de *Logs* similar a la mostrada en la figura 3, donde se logra apreciar el error de memoria insuficiente y que no se eligió un acelerador.

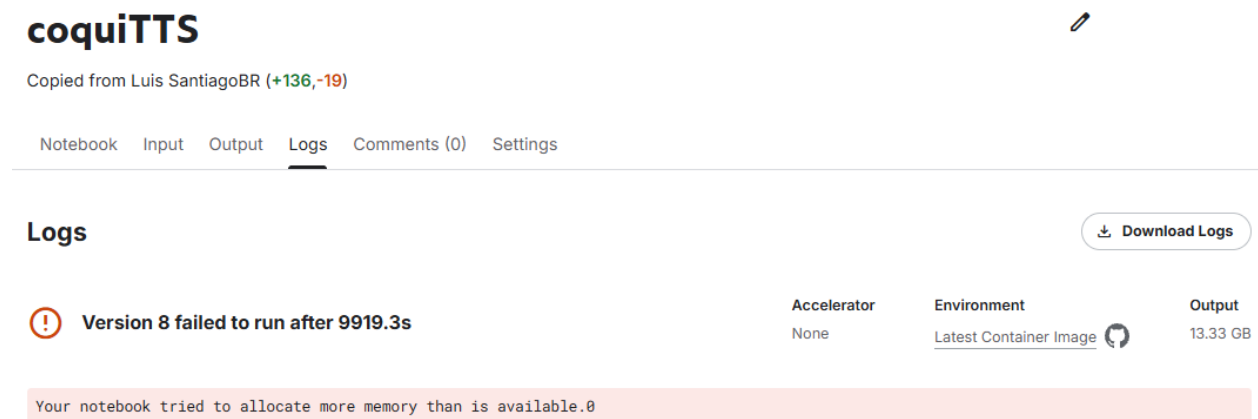


Figura 3: Pestaña de *Logs* del notebook de Kaggle - Error de memoria.

Para evitar este problema, debe asegurarse de haber seleccionado la GPU (T4) como acelerador antes de correr el notebook. Para comprobar que se está ejecutando con una GPU, puede dirigirse a la pestaña de *Logs* del notebook, como se observa en la figura 4, y ver si se tiene un acelerador escogido.

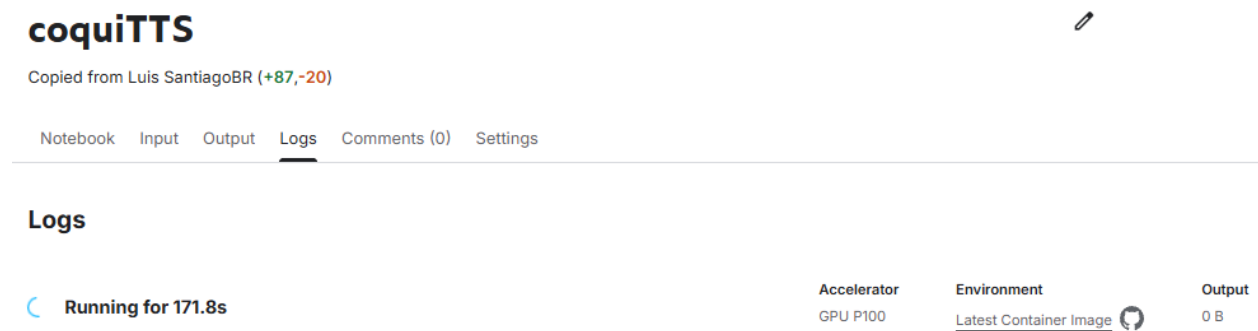


Figura 4: Pestaña de *Logs* del notebook de Kaggle.

2.1.2. Error en organización del metadata.csv

En ocasiones, la transcripción obtenida de la base de datos de voces del TCU puede generar fallas en el archivo `metadata.csv`, porque el archivo `.txt` de donde se transcribió el audio tenía líneas de más o no cumplía un formato adecuado. Lo más simple es corregirlo **antes** de subir el dataset a Kaggle, corriendo localmente el siguiente script sobre la carpeta `dataset` generada por `generar_dataset.py`:

```
import os

# Configuración de rutas (LOCALES, sobre la carpeta antes de comprimirla y subirla a Kaggle)
input_csv = "RUTA_DATASET/metadata.csv"
output_csv = "RUTA_DATASET/metadata_reorganized.csv"

def reorganize_metadata(input_path, output_path):
    with open(input_path, "r", encoding="utf-8") as f:
        lines = f.readlines()

    reorganized = []
    current_entry = ""

    for line in lines:
        line = line.strip()
        if not line:
            continue

        # Detecta si la línea es el inicio de un registro nuevo
        # Busca el patrón: ID|Texto (usualmente empieza por 'voluntario' o tiene un '_')
        if "|" in line and (line.split("|")[0].startswith("voluntario") or "_" in line.split("|")[0]):
            # Si ya tenía un registro acumulado, lo guarda antes de empezar el siguiente
            if current_entry:
                reorganized.append(current_entry)
            current_entry = line
        else:
            # Si no empieza con el formato de audio, es texto huérfano.
            # Se pega al texto de la entrada actual (antes del segundo '|')
            if current_entry:
                parts = current_entry.split("|")
                # Une el fragmento extra al primer bloque de texto
                parts[1] = parts[1].strip() + " " + line
                # Reconstruye la línea (formato 3 columnas: ID|Texto|Texto)
                current_entry = f"{parts[0]}|{parts[1]}|{parts[1]}"
            else:
                current_entry = line

    # Guardar la última entrada
    if current_entry:
        reorganized.append(current_entry)

    # Escribir el archivo final limpio
    with open(output_path, "w", encoding="utf-8") as f:
        for entry in reorganized:
            f.write(entry + "\n")

    # Reemplazar el original
    os.replace(output_path, input_path)
    print(f"Reorganización completada. Se generaron {len(reorganized)} registros válidos.")

# Ejecutar
reorganize_metadata(input_csv, output_csv)
```

Nota: dependiendo de cómo se nombren los archivos `.wav`, es posible que deba modificar el patrón `ID|Texto` usado para detectar el inicio de cada registro.

Si el dataset ya fue subido a Kaggle y prefiere corregirlo dentro del mismo notebook en vez de volver a subirlo, puede agregar una celda nueva justo **antes** de la celda “Ejecutar

todo”, con el siguiente contenido, y luego correr el script anterior apuntando a `/kaggle/working/dataset/metadata.csv` en lugar de una ruta local:

```
import sys
sys.path.insert(0, "scripts")
import pipeline
pipeline.prepare_dataset() # copia el dataset a /kaggle/working/dataset
```

Como `prepare_dataset()` no vuelve a copiar el dataset si ya existe, la celda “Ejecutar todo” reutilizará el `metadata.csv` ya corregido sin sobrescribirlo.

2.1.3. Error por poca cantidad de muestras disponibles

Este error ocurre porque la cantidad de muestras de audio disponibles para el entrenamiento es insuficiente según el tamaño del conjunto de evaluación que usa el pipeline por defecto. El error que aparece en la terminal indica lo siguiente:

```
AssertionError: [!] You do not have enough samples for the evaluation set. You can work
around this setting the 'eval_split_size' parameter to a minimum of 0.04
```

Para arreglar la falla, dentro del notebook copiado, localice la celda que escribe `scripts/train_stage.py` (empieza con `%%writefile scripts/train_stage.py`) y, dentro de la llamada a `GPTTrainerConfig(...)`, agregue el parámetro `eval_split_size`. Por ejemplo:

```
config = GPTTrainerConfig(
    output_path=output_path, model_args=model_args, run_name=run_name,
    ...
    batch_size=batch_size, batch_group_size=48, eval_batch_size=batch_size,
    num_loader_workers=4, eval_split_max_size=256,
    eval_split_size=2,
    ...
)
```

Para la variable `eval_split_size`, si se coloca un valor de porcentaje, se referirá a una cantidad de las muestras disponibles, por ejemplo, 0.04 sería el 4% de la cantidad total de grabaciones. Por otro lado, si se coloca un número entero, por ejemplo, 2, utilizará 2 muestras de audio para evaluar el modelo de voz durante el entrenamiento.